unex

Buscaí.

O *vizinho* que você precisava encontrar.

Victor Rogério Aguiar do Rosario • Lucas Silva Oliveira • Franklin Ferreira dos Santos
Giovanna Salomão Rodrigues • Samira de Jesus Santos • Lucas de Jesus Barreto

01 Descrição da ideia

DESCRIÇÃO GERAL

Um catálogo digital hiper-local.

Moradores encontram prestadores de serviço autônomos da mesma comunidade filtrados por bairro e categoria. Prestadores ganham uma vitrine digital gratuita sem competir com grandes plataformas como O foco é a economia circular dentro do bairro: quem consome e quem oferece dividem o mesmo quarteirão.

O PROBLEMA

Cano estourado às 22h. E agora?

O morador não sabe quem presta serviço perto, depende da sorte de um vizinho conhecer alguém e perde tempo com indicações ocupadas ou distantes.

A SOLUÇÃO

Busca por bairro, contato na hora.

O app mostra prestadores próximos, com portfólio, avaliações de vizinhos e botão direto para o WhatsApp. Sem intermediários, sem taxa por contato.

LOCALIZAÇÃO

Busca por bairro e raio de distância

AVALIAÇÕES

Prova social da comunidade local

WHATSAPP

Contato direto, zero custo operacional

02 Business Model Canvas

Parcerias Chave

- Prestadores de serviços locais (encanadores, eletricistas, manicures, chaveiros, mecânicos, etc.).
- Pequenos comércios do bairro.
- Plataforma de comunicação (WhatsApp).
- Comunidades locais e associações de bairro para divulgação do aplicativo.

Atividades Chave

- Desenvolvimento e manutenção do aplicativo mobile (Flutter)
- Desenvolvimento e manutenção da API backend (NestJS)
- Gestão de autenticação, regras de negócio e dados
- Implementação de sistema de busca, filtros e avaliações
- Evolução contínua do produto (MVP → melhorias)

Recursos Chave

- Aplicativo mobile (Flutter)
- API backend (NestJS - Railway)
- Banco de dados e storage (Supabase)
- Infraestrutura cloud (Railway + Supabase)
- Base de usuários (moradores e prestadores)

Proposta de Valor

- Para o Morador: Praticidade e confiança ao encontrar prestadores de serviço vizinhos via geolocalização, eliminando a demora de indicações informais.
- Para o Prestador: Vitrine digital de baixo custo para formalização e alcance do público local, valorizando o microempreendedor da comunidade.

Relacionamento c/ Clientes

- Interface simples e intuitiva.
- Sistema de avaliações e reputação.
- Contato direto via WhatsApp.
- Experiência focada na comunidade local.

Canais

- Aplicativo mobile (principal canal)
- Grupos de WhatsApp do bairro
- Indicação boca a boca (comunidade local)
- Redes sociais locais

Segmentos de Clientes

- Moradores do Bairro: Pessoas que buscam conveniência, rapidez em emergências domésticas e preferem consumir no comércio local.
- Prestadores de Serviços Autônomos: Encanadores, manicures, chaveiros, técnicos e "faz-tudo" que precisam de presença digital.

Estrutura de Custos

- Desenvolvimento e manutenção do aplicativo
- Infraestrutura cloud (Railway + Supabase)
- Armazenamento de imagens
- Custos com autenticação e backend
- Aquisição de usuários (divulgação local)

Fluxo de Receita

- Assinatura mensal para prestadores (planos de destaque)
- Anúncios e posicionamento privilegiado na plataforma
- Parcerias com comércios locais

03 Requisitos funcionais

Cadastro e autenticação

Perfis separados para morador e prestador, com autenticação integrada ao backend (NestJS) e suporte do Supabase Auth.

Catálogo localizado

Lista de prestadores ordenada por bairro ou raio de distância configurável.

Busca por categorias

Filtros claros: Manutenção, Estética, Automotivo, Limpeza e outras categorias.

Perfis detalhados

Foto, descrição, horários e galeria de trabalhos anteriores.

Sistema de avaliações

Estrelas e comentários de usuários da comunidade.

Contato direto

Botão que abre o WhatsApp via Deep Link.

04 Requisitos não funcionais

Desempenho

Feed fluido com cache local de imagens, paginação no backend e consultas otimizadas.

Portabilidade

Um único código-base em Flutter permite distribuição para Android e iOS, com possibilidade de expansão futura para Web.

Escalabilidade

Arquitetura desacoplada com frontend, API e dados separados, permitindo evolução gradual da infraestrutura.

Custo reduzido no MVP

Uso de Railway e Supabase reduz a complexidade operacional e permite validar o produto com baixo custo inicial.

Manutenibilidade

Autenticação, controle de acesso e regras de negócio centralizados na API NestJS, com suporte do Supabase na gestão de credenciais e dados.

Segurança

O frontend não acessa o banco diretamente; toda comunicação passa pela API, separando responsabilidades.

05 Tecnologias escolhidas

Flutter

FRONTEND (Mobile + Web)

Interface do Usuário (UI), Feed de Postagens, Perfis de Prestadores, Cache de Imagens

NestJs

BACKEND:

Controllers, Services, Regras de Negócio, Filtros

PostgreSQL

BANCO DE DADOS (SUPERBASE)

Banco relacional robusto para usuários, postagens e avaliações.

Supabase Storage

STORAGE DE IMAGENS

Fotos de perfil e portfólio na mesma plataforma do banco.

WhatsApp Deep Link

COMUNICAÇÃO

Abre o WhatsApp do prestador. Zero custo operacional, zero complexidade.

Supabase Auth

AUTENTICAÇÃO

Login seguro com Row Level Security. Gerenciado, sem configuração extra.

06 Modelo de implantação

RAILWAY — BACKEND DEDICADO

API com servidor dedicado

API com servidor persistente (NestJS). Execução contínua, maior controle sobre regras de negócio e melhor previsibilidade de desempenho.

SUPABASE — BAAS

Banco, autenticação e storage gerenciados

Banco de dados PostgreSQL, autenticação e storage gerenciados em uma única plataforma.

RISCO ASSUMIDO

CUSTO PROGRESSIVO DE INFRAESTRUTURA

Escala conforme o crescimento do sistema, em troca de maior controle, estabilidade e performance.

07 Padrões de arquitetura

1

Arquitetura em camadas com back-end dedicado

Frontend → API → Banco, com comunicação desacoplada entre camadas.
Uso de serviços gerenciados (Supabase) combinado com backend próprio (NestJS), garantindo escalabilidade, segurança e flexibilidade.

2

API REST

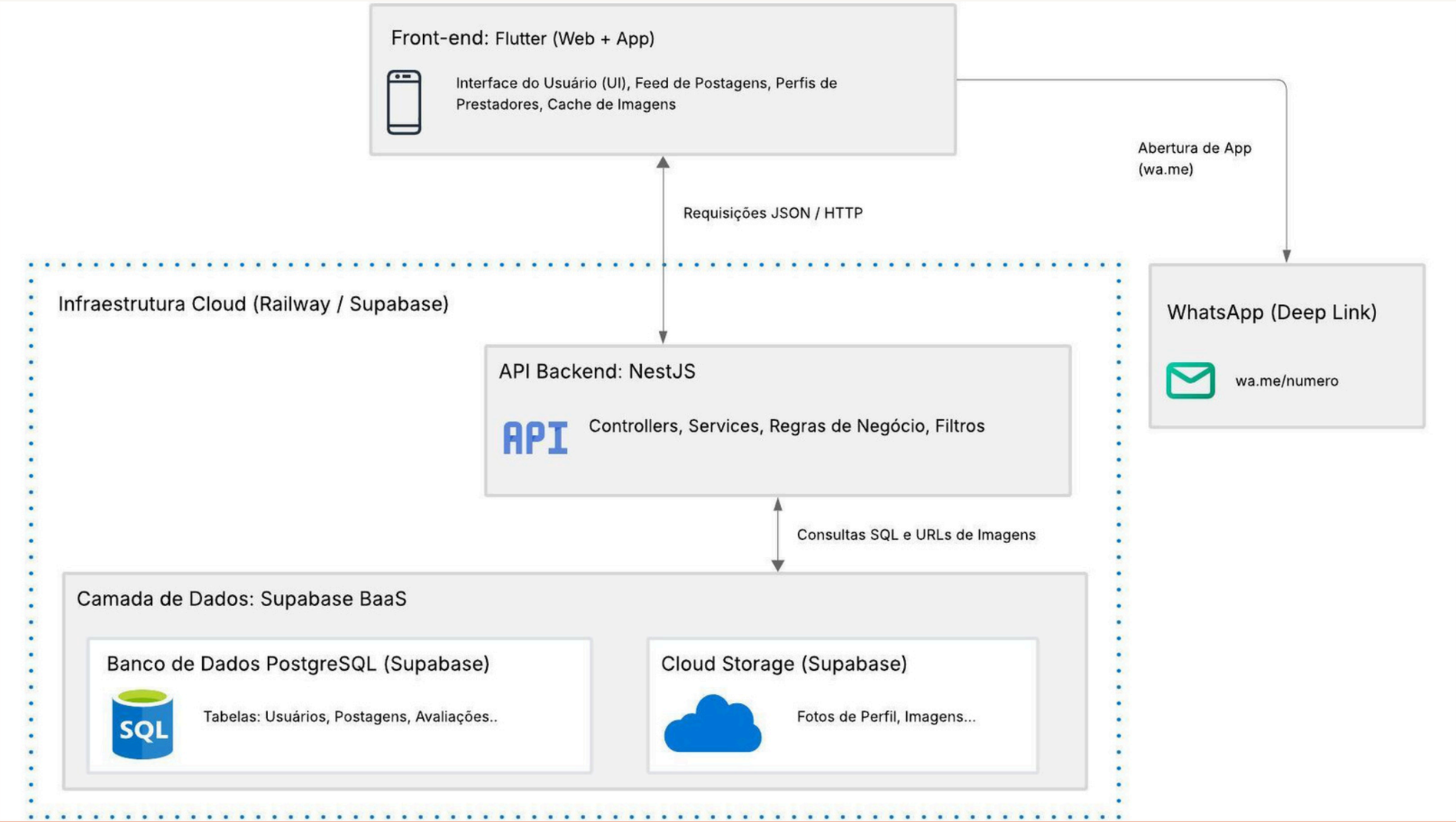
Comunicação via JSON/HTTP. Padrão universal, fácil de testar, documentar e consumir de qualquer cliente.

3

(NestJS) com suporte de BaaS

Backend centralizado em NestJS, responsável pelas regras de negócio e comunicação com o banco.
Uso de BaaS (Supabase) para banco, autenticação e storage.
Arquitetura desacoplada, garantindo segurança, escalabilidade e manutenção facilitada.

08 Diagrama de arquitetura



Buscaí.

Obrigado!